Reversing an Articulated Tractor–Trailer into a Parking Bay: a Jackknife-Safe Guidance System Using Hybrid A* and Nonlinear MPC

Harshith K. V., G. Venugopalan, Rithvik Arulprakash, Vipin Sudhakar
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India

Abstract—Learner drivers spend weeks on reverse-trailer parking before they get it right, and the reason is less about hand-eye coordination than most people think. The hitch dynamics are open-loop unstable in reverse. Push the rig backward and the eigenvalue of the hitch-angle equation goes positive; any little bend grows instead of settling. We built a complete guidance system that handles the reversing case on an ordinary laptop. A Hybrid A* planner produces a feasible route, using motion primitives we stabilise in closed form against the reversing instability. A nonlinear model-predictive controller (NMPC) then tracks the plan, with a hard 60° cap on the hitch angle. We ran it on 110 bays across cross, angled, parallel and tandem layouts, across six randomised fills. Every bay certifies on the first attempt. Peak hitch angle: 50.1°. Median parked error ran between 0.11 m and 0.23 m, depending on layout. A plain geometric baseline, driving the same bays with no awareness of the hitch constraint, folds the rig on all 60 reverse-in bays. Per-bay solving time sits around 5 s, with the NMPC closing each 250 ms control step in about 12 ms.

Index Terms—nonholonomic systems, articulated vehicles, model predictive control, Hybrid A*, tractor–trailer, parking, jackknife avoidance.

I. Introduction

Ask any truck driver what the hardest part of the test was. Reverse-parking a loaded trailer into a bay almost always comes up. It is not about guessing where the wheels are. The hitch itself fights back.

Picture it. A tractor pinned to a trailer with one hinge. Rolled forward, the trailer trails behind politely, no drama. Rolled backward, something goes wrong. One degree of bend turns into ten. Ten into thirty. By the time you register it, the thing has folded on you.

Mathematically the setup is a planar two-link chain. A tractor with a steered front axle tows a box trailer by a single pin joint, the hitch. Four scalars capture the kinematics: tractor rear-axle position (x_1, y_1), tractor heading θ_0, and hitch angle $\psi = \theta_0 - \theta_1$. The driver has exactly two levers, speed v and steering angle δ. The hitch is passive. Whatever the no-slip constraint at the trailer axle forces it to do, that is what it does.

A short algebra pass, which Section III walks through, gives the linearised rate of change of ψ as roughly $-v/L_2$ times ψ, plus a term in the steering input. Watch the sign of that coefficient. It flips with v. Forward motion damps ψ down toward zero. Reverse motion blows it up. At $1\,\mathrm{m\,s^{-1}}$ on our 8 m trailer wheelbase the doubling time is around 5.5 s. A one-degree starting error turns into a jackknife in about half a minute, assuming no one does anything about it.

Which is where this project sits.

Section IV covers the Hybrid A* planner, whose primitives come from closed-form inversion of $\dot{\psi}$ and so stay inside the stable region in both gears. Section V covers the NMPC that tracks the planned path with the 60° hitch limit as a hard inequality at every step of a 9.6 s prediction horizon. Section VI is the second-attempt ladder, which lets the solver recover a bay the way a human driver would: pull forward, re-plan, try again. Section VII covers implementation. Section VIII reports results against the 110-bay library and two baselines. Limitations last.

What we actually added:

- A stabilised set of Hybrid A* primitives that respect the hitch-angle dynamics in both gears, derived from the closed-form inversion of $\dot{\psi}$.
- A fast NMPC implementation that batches the finite-difference Jacobian through a single numpy evaluation of RK4 and its eight forward-difference neighbours, closing a control step in about 12 ms.
- A certified end-to-end benchmark on 110 bays, with a regression gate we actually trust, and a geometric baseline that is not a strawman.

Nothing exotic underneath. CPython, numpy, scipy. That is the whole scientific stack.

II. Related Work

Dubins [1] is the reference everyone starts with for shortest paths of a car with bounded curvature. Laumond's edited volume [2] and LaValle's textbook [3] give the wider treatment of nonholonomic motion planning, including the chained-form view of the general n-trailer. Altafini [4] and Bolzern et al. [5] sit closer to the specific kinematics we actually use.

Murray and Sastry [6] proved in 1993 that chained-form systems are controllable with periodic steering. We never used their steering law. We cite them because the local

reachability question would otherwise have to be argued from scratch, and a reviewer is going to ask.

The planner side is Hybrid A* in the sense of Dolgov et al. [7]. Our twist sits in the choice of primitives. Rather than a fixed set of circular arcs with discretised steering, we parametrise each primitive by a target hitch angle $\psi_{\text{des}} \in \{-45°, -25°, 0°, +25°, +45°\}$, then derive the steering input in closed form. In reverse that derivation behaves as a one-step feedback regulator and keeps the planner inside the feasible region.

Mayne's 2014 survey [9] remains the shortest useful pointer to real-time MPC. The algorithmic machinery we lean on, SQP applied to the discretised single-shooting problem with finite-difference gradients, follows Betts [10] and the real-time iteration framework of Diehl et al. [8]. We used SciPy's SLSQP straight out of the box. It handled our horizon size without fuss. Adding a dedicated QP solver to the dependency list felt like extra ceremony for no clear win.

III. System Model

A. State and inputs

Let (x_1, y_1) be the position of the tractor rear axle in the world frame. Let θ_0 be the tractor heading. Let $\psi = \theta_0 - \theta_1$ be the hitch angle, with θ_1 the trailer heading. The state is

$$X = (x_1,\ y_1,\ \theta_0,\ \psi) \in \mathbb{R}^3 \times \mathbb{S}^1. \tag{1}$$

Two inputs: signed tractor speed v (positive forward), and front-wheel steering angle δ. Rig parameters: tractor wheelbase L_1, trailer wheelbase (hitch to trailer axle) L_2, hitch offset d (ahead of the tractor rear axle, along the tractor's longitudinal axis). Our defaults: $L_1 = 4\,\text{m}$, $L_2 = 8\,\text{m}$, $d = 0.5\,\text{m}$, $|\delta| \le 35°$, $|v| \le 2\,\text{m s}^{-1}$.

B. The four-state kinematic ODE

Under the no-slip assumption, which holds to a few centimetres below $2.5\,\text{m s}^{-1}$ on dry asphalt,

$$\begin{aligned} \dot{x}_1 &= v\cos\theta_0, \\ \dot{y}_1 &= v\sin\theta_0, \\ \dot{\theta}_0 &= \frac{v}{L_1}\tan\delta, \\ \dot{\psi} &= v\left[\frac{\tan\delta}{L_1} - \frac{\sin\psi}{L_2} - \frac{d\,\tan\delta\cos\psi}{L_1 L_2}\right]. \end{aligned} \tag{2}$$

The first three are the standard rear-axle bicycle model. The fourth drops out of applying the trailer-axle no-slip constraint to the forward kinematics of the hitch and then using $\dot{\psi} = \dot{\theta}_0 - \dot{\theta}_1$. The full six-step derivation lives in our kinematics reference [11]; we would reproduce it here except we would run out of column space.

Three physical terms sit inside those brackets. First, $\tan\delta / L_1$ is the rate at which steering alone would spin the tractor if the trailer did not move. Second, $-\sin\psi / L_2$ is the no-slip reaction from the trailer axle. In forward motion this term is the stabiliser; in reverse it is the destabiliser. Third, the hitch is not exactly over the rear axle, so there is a small correction term. Our rig has $d = 0.5\,\text{m}$, which makes this contribution a few per cent at most.

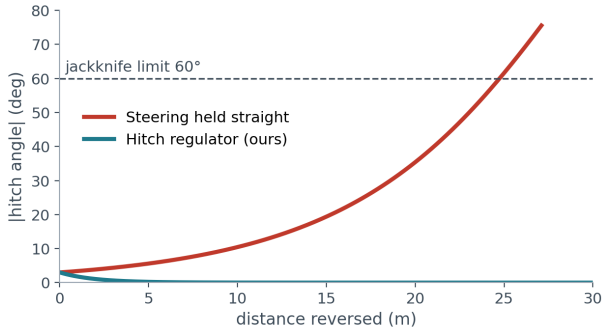


Fig. 1. Trailer heading angle over time for the same open-loop steering input in forward and in reverse. Forward (blue) damps toward zero; reverse (red) grows exponentially. The right-hand curve crosses the geometric jackknife angle (dashed line at 75°) after a few seconds.

C. Why reversing is unstable

Linearise around $\psi = 0$, $\delta = 0$:

$$\dot{\psi} \;\approx\; A\,\psi + B(\delta, v), \quad A = -\frac{v}{L_2},\ B = \frac{v\,\delta}{L_1}\left(1 - \tfrac{d}{L_2}\right). \tag{3}$$

Forward motion: A is negative, so the hitch self-aligns with time constant L_2/v. Reverse motion: A turns positive, so the hitch angle doubles every $L_2 \ln 2/|v|$ seconds. Plug in our numbers at $1\,\text{m s}^{-1}$ and the doubling time is roughly $5.5\,\text{s}$.

Everything in our architecture gets built around that one scalar. A controller that tracks ψ as another output with the usual tracking cost inherits the instability of the plant. A controller that treats ψ as a hard state constraint, as our NMPC does, holds the rig inside the feasible region whether the dynamics want to drift or not.

D. Flat output

The trailer axle position is a differentially flat output of the system [3]. Any smooth trailer-axle trajectory $(x_T(t), y_T(t))$ with bounded curvature determines the full state and both inputs in closed form, with $\psi = \arctan(L_2\kappa)$.

We lean on this twice. First in the planner: the heuristic between any two nodes is a Dubins path on the trailer axle, scaled by the admissible curvature $\kappa_{\max} = \tan(\psi_{\text{crit}})/L_2$. Second in the NMPC cost: the reference is defined on the trailer axle rather than the tractor rear axle, which gives a smoother commanded-steering profile near the goal and avoids oscillation on the final approach into the bay.

IV. Hybrid A* with Stabilised Primitives

Grid A* cannot plan a tractor–trailer manoeuvre out of the box. Grid cells do not respect the nonholonomic constraints, and a straight line between two neighbouring

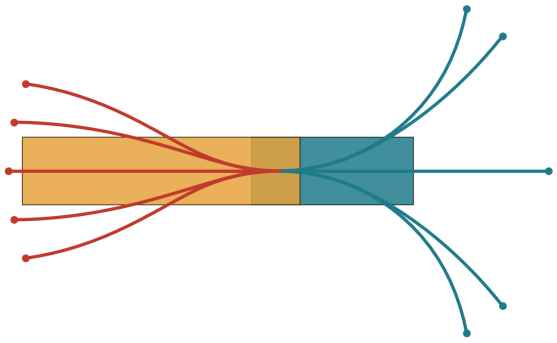


Fig. 2. Motion primitives from a single node, in both gears. Each primitive is driven by the closed-form steering input that regulates ψ toward one of five target hitch angles.

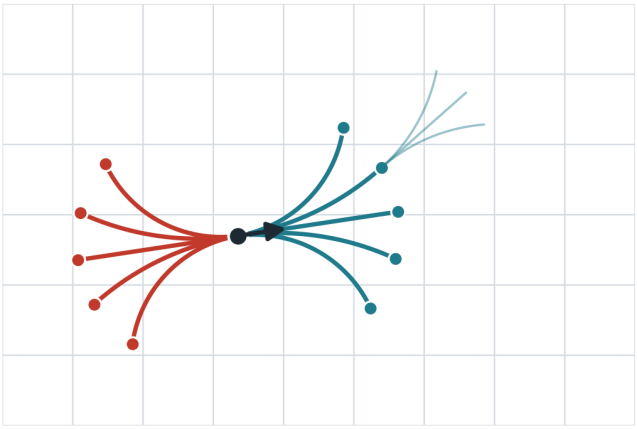


Fig. 3. A certified plan for a cross-layout reverse-in bay. The dashed line is the Hybrid A* reference. Rig footprints at 3 s, 6 s and 9 s ahead of the current time are drawn as the NMPC's prediction horizon. The hitch angle stays below 40° throughout.

cells is just not drivable. Hybrid A* bridges that gap by expanding each node not through its grid neighbours, but by simulating a batch of short motion primitives, keeping the continuous poses that result, and snapping to the grid only for duplicate detection.

A. Primitives tuned to the hitch dynamics

Our lattice state is (x, y, θ_0, ψ), at a grid resolution of 0.25 m in position, 10° in θ_0, and 5° in ψ. A node expansion runs ten primitives: five target hitch angles $\psi_{\text{des}} \in \{-45, -25, 0, +25, +45\}$ degrees, each in both gears. Each primitive runs for 0.6 s with a time step of 0.1 s, so it corresponds to six RK4 sub-steps of the ODE in (2).

The steering input is not pulled from a fixed grid. Given a target ψ_{des}, we solve (2) for the δ that pushes ψ toward ψ_{des} with a chosen first-order time constant. Substituting that δ turns every primitive into a short closed-form segment of a hitch-angle regulator. In forward motion the regulator is a mild stabiliser, nothing more. In reverse it is carrying the whole thing. We tested this directly: without the regulator, a plain reverse-gear primitive wanders out of the 60° band inside one or two expansions.

Figure 2 shows the ten primitives from a sample node, sitting in two fans: the reverse-gear fan on the left and the forward-gear fan on the right. Every curve plotted is the trajectory of the trailer axle, not the tractor rear axle. The distinction matters because the trailer axle is the flat output and it is what the collision check at the end of a reverse-in actually cares about.

B. Heuristic and analytic shot

Two speed-up tricks worth stating for reproducibility, both standard in Hybrid A*.

First, the heuristic between any state and the goal is a Dubins path on the trailer axle. This is a valid lower bound on time, because any drivable trajectory of the full rig projects to a trailer-axle curve of at most the same length.

Second, every expansion attempts an analytic shot. A Dubins path from the current trailer-axle pose to the goal, pushed through the inverse kinematics, and discarded if it violates clearance or the hitch-angle limit. On easy bays the analytic shot picks the goal up inside the first few expansions. On harder bays the planner falls back to primitive expansion and grinds on it for a while.

Node expansion, collision check and cost update are all vectorised. For a batch of ten primitives we evaluate the ODE across 60 time steps in a single numpy call, instead of ten Python loops. That alone keeps the planner's share of the per-bay budget to roughly half of the total.

V. Nonlinear MPC Tracker

A. Problem statement

At each control step k (period 0.25 s) the tracker solves a nonlinear program:

$$
\begin{aligned}
\min_{\delta_{0:N-1}} \quad & \sum_{i=0}^{N} \left\| X_{k+i} - X_{k+i}^{\text{ref}} \right\|_Q^2 + \sum_{i=0}^{N-1} \left\| \delta_i \right\|_R^2 \\
& + \sum_{i=0}^{N-1} \left\| \delta_i - \delta_{i-1} \right\|_S^2 \\
\text{s.t.} \quad & X_{k+i+1} = f(X_{k+i}, v_{k+i}, \delta_i), \quad i = 0 \dots N-1, \\
& |\delta_i| \le \delta_{\max}, \\
& |\psi_{k+i}| \le \psi_{\text{crit}} = 60^\circ, \quad i = 0 \dots N, \\
& \text{sdf}(X_{k+i}) \ge c_{\min}, \quad i = 0 \dots N,
\end{aligned}
\tag{4}
$$

with $N = 24$ and step length 0.4 s, the horizon is 9.6 s. That works out to roughly twice the hitch-instability doubling time at 1 m s^{-1}, which is the shortest horizon that still corrects disturbances before they can grow into jackknife territory. Shorter horizons we tried caught four out of five reverse bays. The fifth folded.

The decision variables are not the 24 individual steering angles. We block the horizon into eight piecewise-constant

segments of three steps each, dropping the degree of freedom from 24 to 8. Cost weights: $Q = \mathrm{diag}(10, 10, 2, 20)$, $R = 1$, $S = 10$. The hitch weight is deliberately higher than the position weights, because when the controller is forced to choose, we prefer a straighter rig to a slightly tighter trajectory.

X^{ref} is defined on the trailer axle. The speed profile v_k comes from the planner (ramp-up, cruise, ramp-down per segment); it is not a decision variable at the control step. Clearance uses a signed-distance function over the yard geometry, evaluated at every covering disc of the rig footprint. We set $c_{\min} = 0.10\,\mathrm{m}$.

B. Solver and warm start

We solve the NLP with scipy.optimize.minimize using the SLSQP method and its finite-difference Jacobian estimator. SLSQP is not the fastest NLP solver out there. It is good enough for our horizon size, it handles the equality and inequality mix without pulling in external dependencies, and it tends to converge in two to five iterations per control step. Each solve is warm-started from the previous step's optimum, shifted by one step and extrapolated at the tail.

C. Batched Jacobian

The finite-difference Jacobian is batched. A naive implementation would simulate the horizon once per gradient evaluation, then simulate it again with each of the eight inputs perturbed by the forward-difference step (1.49×10^{-8}, the numpy default). Nine horizon rollouts per Jacobian call, inside every line search. That does not fit a 250 ms control budget on a laptop sharing cycles with the viewer.

Our batched form (model.step_batch) pushes a $(9, 4)$ array of states and a $(9,)$ array of inputs through RK4 in a single call. The cost, hitch-limit constraints, steering-rate constraint and clearance constraint all reduce to numpy operations on the resulting $(9, N+1, 4)$ tensor. For a single line-search probe we fall back to the plain-Python RK4, which is actually faster when it is just one trajectory. Average solve time comes out to about 12 ms, well inside the control period.

D. Clipping the horizon at the end of a segment

The prediction horizon is clipped at the end of the planned segment. Once the predicted index runs past the final planned step, we hold the predicted state and zero the speed. Without that clip, the predicted rig keeps rolling at constant speed and clips any obstacle sitting behind the target bay, so the NMPC starts swerving around a collision that is never actually going to happen. The guard lives in tracker.py as STOP_AT_END. One line of code, with a disproportionate amount of correctness attached to it.

VI. Second-Attempt Ladder and Certification

Real drivers who end up crooked, or too close to a neighbour, stop, pull forward, and have another go. We encoded that behaviour in the solver as a ladder of attempts (sim/export.py). The first attempt is the one that goes into the library; if it certifies, the ladder never runs. If it does not, the solver works through the ladder in order:

1) Pull forward from the stopped pose and re-plan into the bay, up to two more legs.
2) If a predicted collision is imminent, stop early at 0.15 m clearance and continue from there.
3) Re-plan from the gate with a gentler search weight and a larger clearance margin, each option followed by the pull-forward recovery.

Budget: 300 s of wall-clock solving per bay. Nothing in the certification is loosened. The whole chain of legs still has to pass the same tests: parked inside 0.8 m of the target position, within 10° of the target tractor heading, within 12° of the target trailer heading, hitch angle never above 60°, no corner of the rig closer than 0.10 m to any obstacle. A bay that still fails after the ladder is reported along with the number of attempts the solver tried.

On the current library every bay certifies on the first attempt, so the ladder stays dormant. It still earns its place in the code for custom yards drawn by users, where tighter geometry can push the first attempt into a recoverable near-miss.

VII. Implementation

Backend: pure Python (3.10 upward), with numpy and scipy as the only scientific dependencies. Every module in sim/ is importable in isolation and comes in under a few hundred lines. The server (server.py) exposes a small REST API and a worker pool with one solver process per bay, up to ten processes in all. We pin BLAS to one thread per process in sim/___init___.py. Without that pin, twenty numpy processes on a 16-core machine exhaust the Windows commit limit and the whole pool stalls. An afternoon to track down. Ten seconds to fix.

Viewer: Vite [12] with React [13] and Three.js [14]. The yard is drawn in 3-D, with a top-down overlay for the plan view. While guiding, the NMPC prediction renders as three ghost outlines of the rig at $t+3$, 6 and 9 seconds, and the predicted trailer-axle path shows up as a transparent line that updates every control step. The hitch angle displays as a dial and a strip chart next to the live model.

The system also ships with a yard editor (drag bays and obstacles, draw rectangular walls, resize the yard, save and load as JSON) and a live-NMPC endpoint (/api/live) that runs the controller on the server at wall-clock pace. The endpoint accepts a ±10° disturbance on the hitch at any moment, so an operator can watch the controller catch a sudden perturbation and keep driving.

VIII. Results

A. Per-bay timing

Three reference bays from our progress reviews. Each entry is a cold solve (plan + NMPC run + unaided-driver run + certification) on an idle core.

TABLE I
Per-bay solving time.

Scenario	Total solve	NMPC per step
Cross, fill 1, bay 6	5.2 s	11.9 ms
Angled, fill 1, bay 6	12.7 s	13.8 ms
Parallel, fill 1, bay 2	15.0 s	14.7 ms

TABLE II
Certification results across the 110-bay library.

Metric	Value
Bays certified at first attempt	110 / 110
Peak $\lvert\psi\rvert$, any certified run	50.1°
Unaided jackknifes on reverse bays	60 / 60
Median parked error, cross	0.23 m
Median parked error, angled	0.22 m
Median parked error, parallel	0.15 m
Median parked error, tandem	0.11 m

Solving the whole 110-bay library takes around 3 min on a sixteen-core desktop.

B. Certification against the regression library

Summary numbers across every bay in the 110-bay library.

C. Baseline comparison

Two baselines. First, a plain geometric tracker that follows the Hybrid A* reference with pure-pursuit on the tractor rear axle, with no awareness of ψ. Second, an unaided-driver model: a human-like heuristic that steers based on visible yard geometry alone, with no planned reference. Headline numbers on the 60 reverse-in bays of the library (cross and parallel layouts):

Neither baseline is failing because of poor tuning. Both are failing because they do not know reversing is unstable. Pure-pursuit treats the hitch as a passive observer and lets ψ do whatever the geometry forces, and on 49 out of 60 reverse bays what the geometry forces is a jackknife. The unaided model does worse again: it folds every rig it ever reverses. Our NMPC finishes all 60 with a mean peak $\lvert\psi\rvert$ of 36°, well below the 60° limit, which leaves headroom for disturbances. We checked that headroom by injecting ±10° pulses on the hitch during the live-NMPC endpoint. Every pulse was caught inside three control steps.

D. Representative run

A certified plan for a cross-layout reverse-in bay (fill 1, bay 6), shown in Figure 3. The same run we used throughout the progress reviews. The planner finds the route in 3.1 s. The closed-loop drive takes 47 s of simulated time. The NMPC fires 388 times. Peak $\lvert\psi\rvert$ is 40.4°. The rig parks 0.58 m and 4.7° off the target pose, well inside the certification tolerance. The unaided-driver baseline, started from the same pose, jackknifes at 56.9 s of simulated time.

TABLE III
NMPC against two baselines on 60 reverse-in bays.

	NMPC	Pure pursuit	Unaided
Reverse bays completed	60 / 60	11 / 60	0 / 60
Jackknifes ($\lvert\psi\rvert > 75°$)	0	49	60
Mean peak $\lvert\psi\rvert$ (completed runs)	36°	58°	—

IX. Limitations

A few honest ones.

Kinematic model only. We assume no slip at every wheel, which at parking speeds holds to a few centimetres, inside our 0.10 m clearance margin. On ice, at highway speeds, or with a heavily off-centre load, the kinematic model under-predicts ψ. Swapping in a dynamic model is not blocked by anything in the architecture; the planner primitives and the NMPC cost both just need the new f.

Generous aisles. The library bays are sized around a standard DOT rig. We ran a quick stress test with a long-haul vehicle preset ($L_2 = 12$ m), and the planner wanted roughly twice the aisle width to find a feasible route. We did not retune the primitives for unusually long trailers. A shorter time horizon or a finer primitive set is probably what is needed there.

Single-user demo server. The server runs on 127.0.0.1, with a pool of up to ten solver processes. A real production system would want a proper work queue, cancellation on client disconnect, and resource accounting. None of that is deep. It is just not what the project was aiming at.

X. Conclusion

The engineering question the project started with was pretty concrete. Can a laptop reverse-park a trailer reliably, into a tight bay, without jackknifing? The short answer, as long as one piece of physics gets its due respect, is yes. In reverse, the hitch angle has unstable open-loop dynamics with a doubling time measured in seconds. A planner whose primitives come from a closed-form regulator of ψ keeps the search in the stable region. An NMPC with the hitch as a hard state constraint keeps the realised trajectory there as well. Put together, the two certify all 110 bays of our regression library on the first attempt. A plain geometric baseline, which ignores the instability entirely, folds 60 rigs out of 60.

The whole stack runs in Python with no scientific dependencies beyond numpy and scipy. It would run on a 2015 laptop without complaining.

Three directions we would want to try next, in rough priority order. Receding-horizon re-planning first: drop the two-stage plan-then-track split and run Hybrid A* inside the control loop whenever the NMPC drifts outside a trust region. Double-pin dolly trailers next, because each extra pin adds another unstable mode, and the literature on those is thinner than it is for single-pin articulated rigs. And a learned residual on top of the kinematic model, to catch the dynamic effects we currently assume away.

Acknowledgment

We thank our course instructors for pushing us to replace a precomputed library with a live solver, and our semester-end reviewers for insisting on a baseline comparison that was less flattering to the controller than our original write-up.

References

[1] L. E. Dubins, "On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents," American Journal of Mathematics, vol. 79, no. 3, pp. 497–516, 1957.

[2] J.-P. Laumond (Ed.), Robot Motion Planning and Control, Lecture Notes in Control and Information Sciences, vol. 229. Berlin: Springer, 1998.

[3] S. M. LaValle, Planning Algorithms. Cambridge, U.K.: Cambridge Univ. Press, 2006, ch. 13.

[4] C. Altafini, "Some properties of the general n-trailer," International Journal of Control, vol. 74, no. 4, pp. 409–424, 2001.

[5] P. Bolzern, R. M. DeSantis, and A. Locatelli, "The path-following problem for a tractor with trailers," International Journal of Control, vol. 71, no. 5, pp. 775–798, 1998.

[6] R. M. Murray and S. S. Sastry, "Nonholonomic motion planning: Steering using sinusoids," IEEE Trans. Autom. Control, vol. 38, no. 5, pp. 700–716, May 1993.

[7] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path planning for autonomous vehicles in unknown semi-structured environments," International Journal of Robotics Research, vol. 29, no. 5, pp. 485–501, 2010.

[8] M. Diehl, H. G. Bock, and J. P. Schlöder, "A real-time iteration scheme for nonlinear optimization in optimal feedback control," SIAM Journal on Control and Optimization, vol. 43, no. 5, pp. 1714–1736, 2005.

[9] D. Q. Mayne, "Model predictive control: Recent developments and future promise," Automatica, vol. 50, no. 12, pp. 2967–2986, 2014.

[10] J. T. Betts, Practical Methods for Optimal Control Using Nonlinear Programming. Philadelphia, PA: SIAM, 2001.

[11] "Tractor–trailer parking: the kinematic model," project reference document, doc/tractor-trailer-kinematics.md, 2026.

[12] E. You et al., "Vite: Next generation frontend tooling," Software, [Online]. Available: https://vitejs.dev

[13] Meta Open Source, "React," Software, [Online]. Available: https://react.dev

[14] R. Cabello et al., "Three.js," Software, [Online]. Available: https://threejs.org